Appendix C: IEEE 830 Template

Software Requirements Specification

for

Gym Management System

Version 0.1

Prepared by Jaime Esteban Reaño

IFE – Software Engineering

04/03/2026

«Page Break»

Table of Contents

«Table of Contents»

Revision History

Jaime Esteban Reaño	04/03/2026	Initial version of the SRS document created. Project topic selected: Gym Management System.	0.1

«Page break»

1. Introduction

1.1. Purpose

This document specifies the software requirements for the Gym Management System, a software solution intended to support the daily management of a gym. The system will provide functionalities for member registration, class booking, trainer management, membership tracking, and payment control. This specification defines the current scope of the project and will be progressively updated as new analysis and design artifacts are developed during the course.

1.2. Document Conventions

Functional requirements are identified using the prefix FR, while non-functional requirements are identified using the prefix NFR. All mandatory requirements are written using the word “shall” to ensure precision and verifiability. UML diagrams included in the document are used as analysis and design support artifacts.

1.3. Intended Audience and Reading Suggestions

This document is intended for course instructors, project team members, testers, and future developers of the Gym Management System. Readers should begin with the Introduction and overall description sections to understand the purpose and scope of the project, and then continue with the system features and interface requirements

1.4. Product Scope

This Gym Management System aims to digitize and simplify the management of gym operations. Its main objective is to centralize information related to members, trainers, fitness classes, memberships, and payments in a single system. The expected benefits include improved administrative efficiency, reduced manual errors, easier scheduling, and better service for gym members

1.5. References

- Lab 1: Requirements Analysis – Software Engineering course materials

- Lab 2: UML Use Case Diagrams – Software Engineering course materials

2. Overall Description

2.1. Product Perspective

This Gym Management System is conceived as a new standalone software product. It is intended to replace informal or manual management methods based on spreadsheets, paper records, or disconnected applications. The system will centralize core gym operations in a single platform.

2.2. Product Features

«Summarize the major features the product must perform or must let the user perform. Details will be provided in Section 3, so only a bullet list is needed here. Organize the features to make them understandable to any reader of the specification. A picture of the major groups of related requirements and how they relate, such as a top level data flow diagram or object class diagram, is often effective.»

- «Thing 1»
- «More things as required»

2.3. User Classes and Characteristics

«Identify the various user classes (intended target for the product) that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.»

2.4. Operating Environment

«Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.»

2.5. Design and Implementation Constraints

«Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer’s organization will be responsible for maintaining the delivered software).»

2.6. User Documentation

«List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.»

2.7. Assumptions and Dependencies

«List any assumed factors (as opposed to known facts) that could affect the requirements stated in the specification. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).»

3. System Features

«The purpose here is to give a brief introduction to the system features, so that §4 has something to reference. Then, the full explanation of the system features is given in §5, likely referencing §4. The numbering in §3 and §5 should match.»

3.1. «Feature Name»

«Brief description of the feature, generally 1-3 sentences»

3.«N». «Additional feature blocks as needed»

4. External Interface Requirements

4.1. User Interfaces Overview

«Describe the high level functionality of the system from the user's perspective. Explain how the user should be able to use your system to complete all the expected features and the feedback information that will be displayed for the user (for each screen the user will see). For example, specify whether it is GUI or text based interface and how at high level it would work. Discuss at high level how user will interact with the game such as clicking buttons, typing in some selection character, etc.»

«Describe the minimum requirements for the interface. This may include some sample screen images whether for text or GUI approach, any GUI standards or product family style guides that are to be followed, screen layout constraints, for GUI standard buttons and functions (e.g., help) that must appear on every screen, keyboard shortcuts, error message display standards, and so on»

4.2. Hardware Interfaces

«Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.»

4.3. Software Interfaces

«Describe the connections between this product and other specific software components (name and version), including databases, interpreters, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.»

5. System Features/Modules

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

5.1. «System Feature Name»

5.1.1 Description and Priority

«Provide a short description of the feature. Give its priority, too.»

5.1.2 Stimulus/Response Sequences

«"Stimulus" or "Condition"»: «Describe the stimulus or condition»

Response: «Describe the response»

«Additional Stimulus/Response pairs as needed»

5.1.3 Functional Requirements

«Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous, verifiable, and necessary. Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.»

REQ-1.1: «Some sentence goes here, in "shall" form, usually something like this: "Upon <some event or condition>, the <system, or the name of a module> shall <perform some action>." Note that the entire requirement must be stated in the requirements area. Everything else is just a lead-up to this. There should be one such entry per S/R pair.»

REQ-1.«N»: «Additional S/R-related requirements as needed.»

REQ-1.«N»: «List any additional requirements needed in "Shall" form, but not necessarily in "Upon/Shall" form.»

REQ-1.«N»: «Additional requirements as needed.»

5.«N». «Additional feature blocks»

6. Nonfunctional Requirements

6.1. Performance

NF-1.1: «Any performance-related issue»

NF-1.«N»: «Any performance-related issue»

6.2. Security

NF-2.1: «Any security-related issue»

NF-2.«N»: «Any security-related issue»

6.«N». «Any additional "nonfunctional" blocks»